UTN
Facultad Regional Córdoba

TUP

Tecnicatura Universitaria en Programación

PROGRMACIÓN III

Back End – Front End

Trabajo Práctico Integrador:

Pokémon TCG

TPI

2° Año – 3° Cuatrimestre

CC BY ND NC

Introducción 2

Objetivos 3

Requerimientos funcionales 4

RF-01) Reglas del juego ... 4
a) Preparación de la partida: ... 4
b) Estructura del turno: ... 4
c) Sistema de ataque — secuencia de resolución: ... 5
d) Proceso de Knockout: ... 5
e) Condiciones especiales: ... 6
f) Condiciones de victoria y derrota: ... 6
RF-02) Tipos de cartas ... 7
RF-03) Gestión del juego ... 8
RF-04) Construcción de mazos ... 9
RF-05) Guardado y persistencia del estado ... 10
RF-06) Comunicación en tiempo real ... 10
RF-07) Interfaz de usuario ... 11

Requerimientos No Funcionales 12

RNF-01) Rendimiento y optimización ... 12
RNF-02) Calidad del código y buenas prácticas ... 12
RNF-03) Pruebas unitarias y cobertura de código ... 13
RNF-04) Diseño de software y patrones de diseño ... 13
RNF-05) Seguridad ... 14
RNF-06) Usabilidad y experiencia de usuario ... 14
Tecnologías Obligatorias ... 15
Forma de trabajo ... 15
Entregables obligatorios ... 15
Forma y fechas de entrega ... 16
Evaluación y Calificación ... 16

Apéndice — Material de referencia 18

Reglamento oficial del juego ... 18
API de cartas ... 18
Cliente digital oficial de referencia ... 18
Tecnologías utilizadas — documentación oficial ... 18
Control de versiones ... 18

Introducción

El **Pokémon Trading Card Game (TCG)** es uno de los **juegos de cartas coleccionables** más populares del mundo, en el que **dos jugadores** se enfrentan con **mazos de 60 cartas** construidos estratégicamente. Cada jugador dispone de **Pokémon** para combatir, **Energías** para potenciarlos y **cartas de Entrenador** para obtener **ventajas tácticas**. El objetivo es vencer a los **Pokémon del oponente** para acumular **cartas de Premio**, o **agotar el mazo contrario**, o dejar al rival **sin Pokémon en juego**.

En este contexto, como parte de la materia **Programación III**, se presenta el desafío de desarrollar una **versión digital funcional** del **Pokémon TCG**, implementando tanto el **front-end** como el **back-end** del juego. Este proyecto pondrá a prueba los **conocimientos adquiridos en programación**, requiriendo una **comprensión profunda** de las **reglas oficiales del juego** (basadas en el **XY1 Rulebook**), habilidades en **diseño de interfaces**, **manejo de datos en tiempo real** y **lógica de negocio compleja**.

Los estudiantes deberán aplicar **conceptos clave de desarrollo de software**, tales como **arquitectura cliente-servidor**, comunicación a través de **APIs REST**, **gestión del estado del juego en tiempo real** y **sincronización de eventos entre jugadores**. Adicionalmente, integrarán la **API pública pokemontcg.io (v2)** para obtener los **datos oficiales de las cartas**, implementando un **sistema de caché local** para garantizar la **disponibilidad** y el **rendimiento**.

Este desafío ofrece una **oportunidad única** para integrar **conocimientos teóricos** con su **aplicación práctica** en un **proyecto realista y motivador**, fomentando la **creatividad**, el **trabajo en equipo** y las **buenas prácticas de la industria del software**.

Objetivos

El objetivo principal de este trabajo integrador es aplicar los conocimientos adquiridos durante las clases de Programación III para desarrollar el juego **Pokémon TCG** completamente funcional en **Java con Spring Boot (Back-End)** y **Angular (Front-End)**. Esto implica diseñar e implementar una aplicación que permita a dos jugadores competir entre sí en partidas en tiempo real, siguiendo todas las reglas oficiales del juego según el reglamento **XY1** (Rulebook oficial de Pokémon TCG), e integrando las cartas reales a través de la API pública pokemontcg.io.

El juego desarrollado debe incluir características avanzadas que demuestren la comprensión y aplicación de los conceptos de programación aprendidos, como la implementación de patrones de diseño, la gestión eficiente del estado del juego, la modularidad y la reutilización de código, la manipulación de estructuras de datos complejas, y la implementación de comunicación en "tiempo real". También deben aplicarse los conocimientos adquiridos sobre gestión del código fuente, su versionado y manipulación mediante **Git** y **GitHub** con el **workflow GitFlow**.

El desarrollo requerirá la aplicación de conocimientos de programación orientada a objetos en **Java 21**, manejo de estructuras de datos, diseño de **APIs REST**, gestión de bases de datos relacionales (PostgreSQL o MySQL), y el uso de herramientas como **Spring Boot, Maven, JUnit, Mockito, Angular 21+, RxJS,** entre otros.

Este proyecto no solo permitirá a los estudiantes aplicar los conceptos y técnicas aprendidas en el aula, sino que también los desafiará a enfrentarse a situaciones reales de desarrollo de software, como la planificación, diseño, implementación, pruebas y entrega de una aplicación de software completa y funcional.

Adicionalmente, los alumnos deberán construir la interfaz de juego como un tablero interactivo con sistema de drag & drop, donde se visualicen en tiempo real todos los elementos del juego: Pokémon en campo (Activo y Banca), energías unidas, contadores de daño, condiciones especiales, mano del jugador, cartas de Premio restantes y el log de acciones.

Requerimientos funcionales

La siguiente lista de requerimientos debe cumplirse para que el trabajo se considere completo y correcto, a excepción de los requerimientos opcionales, que serán considerados como parte de la nota normativa para aspirar a una posible promoción.

RF-01) Reglas del juego

Define el núcleo del motor de juego. Es el requerimiento más crítico del proyecto: cubre la preparación de la partida, el ciclo completo de un turno, la resolución de ataques, el proceso de knockout, las condiciones especiales y las condiciones de victoria y derrota. Todo debe ser validado y ejecutado exclusivamente en el backend.

a) Preparación de la partida:

- Ambos jugadores barajan sus mazos y roban **7 cartas** iniciales.
- **Mulligan (Segunda Oportunidad):** si un jugador no tiene ningún Pokémon Básico en su mano inicial, debe mostrar su mano al rival, barajar y volver a robar 7 cartas. Por cada mulligan que declare un jugador, su oponente puede robar **1 carta adicional**. Se repite hasta que ambos tengan al menos un Básico.
- Cada jugador coloca su **Pokémon Activo** boca abajo y hasta **5 Pokémon Básicos en Banca**, también boca abajo.
- Cada jugador toma las primeras **6 cartas** de su mazo y las coloca boca abajo como sus **cartas de Premio**.
- Se lanza una moneda para determinar quién comienza. Ambos jugadores revelan sus Pokémon y comienza la partida.

b) Estructura del turno:

- **Fase 1 – Robo obligatorio:** el jugador roba **1 carta** del mazo. El jugador que empieza **no roba en su primer turno**. Si al intentar robar el mazo está vacío, ese jugador **pierde**.
- **Fase 2 – Acciones** (en cualquier orden, todas opcionales): colocar Pokémon Básicos en Banca, evolucionar Pokémon con las restricciones correspondientes (**no en el turno en que entró en juego**, **no en el primer turno del jugador**), unir **1 carta de Energía por turno** a cualquier Pokémon propio, jugar cartas de Entrenador (**Objetos sin límite**; **1 Partidario por**

turno; **1 Estadio por turno**), retirar el Pokémon Activo pagando su costo de retirada (**1 vez por turno**), usar Habilidades de Pokémon propios.

- **Fase 3 – Ataque:** el jugador puede atacar con su Pokémon Activo. **No disponible en el primer turno del jugador que empieza.** El ataque **finaliza el turno** automáticamente.

c) Sistema de ataque — secuencia de resolución:

1. El jugador anuncia el ataque y verifica que su Pokémon Activo tenga la **Energía requerida**.
2. Si el Pokémon Activo está **Confundido**, lanzar moneda: cruz → el ataque **falla**, el atacante recibe **3 contadores de daño** y el turno termina.
3. El jugador realiza todas las **selecciones** que exija el ataque (elegir objetivo, etc.).
4. Se ejecutan los **requisitos previos** del ataque (lanzamientos de moneda indicados en el texto de la carta, etc.).
5. Se aplican efectos que puedan **modificar o cancelar** el ataque (efectos de ataques del turno anterior del rival, etc.).
6. **Cálculo y aplicación del daño:**
   - Daño base indicado en la carta.
   - Modificadores por efectos de Entrenadores u otros efectos activos sobre el atacante.
   - Aplicar **Debilidad** del defensor (**×2** al daño).
   - Aplicar **Resistencia** del defensor (**−20** al daño). Mínimo: 0.
   - Modificadores por efectos activos sobre el defensor.
   - Colocar **1 contador de daño** por cada **10 puntos** de daño resultantes.
7. Aplicar **efectos posteriores al daño:** condiciones especiales infligidas, descartes de Energía, daño a Pokémon en Banca, curación, etc.

d) Proceso de Knockout:

- Un Pokémon queda **Fuera de Combate** cuando sus contadores de daño × 10 **≥** sus HP.
- El Pokémon y **todas las cartas unidas** a él se descartan.
- El oponente toma **1 carta de Premio** (o **2 cartas de Premio** si el Pokémon derrotado es un **Pokémon-EX**).
- El dueño debe reemplazar el Pokémon Activo con uno de su Banca. Si **no tiene Pokémon en Banca, pierde la partida**.

e) Condiciones especiales:

- **Dormido:** el Pokémon **no puede atacar ni retirarse**. Entre turnos, lanzar moneda: cara → despierta; cruz → sigue dormido.
- **Quemado:** entre turnos, lanzar moneda: cruz → **2 contadores de daño** sobre el Pokémon Quemado.
- **Confundido:** al intentar atacar, lanzar moneda: cruz → el ataque **falla** y el atacante recibe **3 contadores de daño**.
- **Paralizado:** el Pokémon **no puede atacar ni retirarse**. Se cura automáticamente **entre turnos** al final del turno en que fue paralizado.
- **Envenenado:** entre turnos se coloca automáticamente **1 contador de daño** (sin lanzar moneda).

Gestión de incompatibilidades y procesamiento entre turnos:

- **Dormido, Confundido y Paralizado** son **mutuamente excluyentes**: la condición más reciente reemplaza a la anterior.
- **Quemado y Envenenado** usan marcadores independientes y **pueden coexistir** entre sí y con cualquiera de las otras tres. Un Pokémon puede estar **Quemado + Envenenado + Paralizado** simultáneamente.
- El procesamiento entre turnos sigue este **orden fijo: (1) Envenenado → (2) Quemado → (3) Dormido → (4) Paralizado**. Luego se verifican efectos de Habilidades y se comprueba si algún Pokémon quedó Fuera de Combate.
- Todas las condiciones especiales se eliminan cuando el Pokémon **se retira a la Banca o evoluciona**.

f) Condiciones de victoria y derrota:

- **Victoria por Premios:** un jugador toma su **última carta de Premio**.
- **Victoria por knockout total:** el oponente **no tiene ningún Pokémon** en el campo para reemplazar al que fue derrotado.
- **Derrota por mazo vacío:** al intentar robar al inicio del turno, el **mazo está vacío**.
- **Muerte Súbita:** si ambas condiciones de victoria se cumplen simultáneamente, se inicia una nueva partida con **1 sola carta de Premio** cada uno. Se repite hasta que haya un ganador.

RF-02) Tipos de cartas

El sistema debe reconocer, modelar y aplicar correctamente las reglas de cada tipo de carta del juego. Cada tipo tiene comportamiento, restricciones y ciclo de vida propios que el motor de juego debe respetar.

- **Pokémon:** cartas con HP, ataques, debilidad, resistencia y costo de retirada. Pueden ser **Básico**, **Fase 1** o **Fase 2**. Las cartas de Fase 1 y Fase 2 son cartas de evolución: no se puede evolucionar en el primer turno en que el Pokémon entra en juego, ni en el primer turno del jugador.
- **Pokémon-EX:** Pokémon Básico con mayor HP y ataques más poderosos. Regla especial: cuando un Pokémon-EX queda Fuera de Combate, el oponente toma **2 cartas de Premio** en lugar de 1. El sufijo **-EX** forma parte del nombre del Pokémon.
- **Energía Básica:** sin límite de copias en el mazo; se puede unir **1 por turno** a cualquier Pokémon en juego.
- **Energía Especial: máximo 4 copias** en el mazo; con efectos adicionales según el texto de la carta.
- **Entrenador – Objeto:** se puede jugar **ilimitadas veces** por turno; efecto inmediato y luego se descarta.
- **Entrenador – AS TÁCTICO:** subtipo especial de Objeto de alto impacto. Restricción: solo puede haber **1 carta de AS TÁCTICO en todo el mazo**, sin importar cuáles sean. El Deck Builder debe validar esta restricción.
- **Entrenador – Partidario: solo 1 por turno**; efecto inmediato y luego se descarta.
- **Entrenador – Estadio: solo 1 por turno**; permanece en juego (zona compartida); reemplaza al Estadio anterior si lo hubiera.
- **Entrenador – Herramienta Pokémon:** se une a un Pokémon (**máximo 1 por Pokémon**); permanece hasta que el Pokémon sea descartado.
- ***(Opcional)* Pokémon Megaevolución:** evolución de un Pokémon-EX. Regla especial: al convertir un Pokémon en Megaevolución, **el turno del jugador termina inmediatamente**. Al quedar Fuera de Combate, el oponente toma **2 cartas de Premio**. Su implementación será considerada mejora y sumará puntos extra en la evaluación.

RF-03) Gestión del juego

Define el ciclo de vida completo de una partida: desde su creación hasta la detección del ganador. Establece los estados del sistema, la responsabilidad exclusiva del backend como fuente de verdad, y los requisitos de persistencia y trazabilidad en tiempo de ejecución.

- Gestión del ciclo completo de una partida: creación, **matchmaking entre dos jugadores**, gestión de turnos y su orden, y detección de la **condición de finalización**.
- Gestión de los **estados del juego:**
  - **WAITING** — esperando que un segundo jugador se una.
  - **SETUP** — preparación del tablero (mulligan, colocación inicial, cartas de Premio).
  - **ACTIVE** — partida en curso; incluye el ciclo de turnos con sus fases (**DRAW** → **MAIN** → **ATTACK** → **BETWEEN_TURNS**).
  - **FINISHED** — partida finalizada con un ganador determinado.
- El **backend es la única fuente de verdad** del estado del juego en todo momento. El frontend es únicamente una capa de presentación.
- El juego debe mantener un **log completo** de todos los eventos y acciones de cada partida (turno, jugador, acción, resultado), utilizable para auditoría, revisión y reconexión.
- **Persistencia automática** del estado completo del juego luego de cada acción relevante, suficiente para reconstruir la partida ante cualquier eventualidad o desconexión.
- Los datos de cartas se obtienen del **caché local en base de datos** (cargado por el Deck Builder vía pokemontcg.io). El motor del juego **no debe realizar llamadas directas a la API externa** durante una partida.
- ***Opcional:*** chat de comunicación entre los dos jugadores durante la partida.
- ***Opcional:*** sistema de ranking o historial de partidas por jugador.

RF-04) Construcción de mazos

Cubre la implementación del Deck Builder: interfaz para armar mazos, integración con la API de cartas, validación según las reglas oficiales y persistencia por jugador. Es el punto de entrada al juego y el único lugar donde se consume la API externa.

- **Restricción de expansiones (base obligatoria):** el Deck Builder debe trabajar con el set **XY** (*XY Unlimited*, código **xy1**, lanzamiento 05/02/2014, **146 cartas**) como expansión base. Todas las funcionalidades de construcción, validación y juego deben funcionar correctamente con las cartas de este set. *Si un equipo incorpora expansiones adicionales y el juego funciona correctamente con ellas, se considerará una **mejora opcional** con puntaje extra. No es un requisito*.
- Implementar un **Deck Builder** que permita buscar cartas de la API pokemontcg.io (v2) **filtrando por el set xy1**, agregar cartas al mazo y gestionar los mazos guardados del jugador.
- **Validación del mazo** según las reglas oficiales:
  - **Exactamente 60 cartas.**
  - **Máximo 4 copias** del mismo nombre (excepto Energía Básica, **sin límite** de copias).
  - **Máximo 1 carta de AS TÁCTICO** en todo el mazo (sin importar cuál sea).
  - **Al menos 1 Pokémon Básico.**
- **Persistencia de mazos** en base de datos: crear, editar, eliminar y listar mazos por jugador.
- Interfaz visual clara que muestre: **contador de cartas (0/60)**, límites de copias por carta, y **errores de validación** con mensajes descriptivos y accionables.

RF-05) Guardado y persistencia del estado

Especifica qué datos deben persistirse y con qué nivel de detalle, de forma que la partida pueda reconstruirse completamente ante cualquier falla o desconexión.

- El estado persistido después de cada acción relevante debe incluir como mínimo: **estado del tablero** (Pokémon Activo y Banca de cada jugador con sus cartas unidas), **manos de ambos jugadores**, **mazos** (orden incluido), **pilas de descarte**, **cartas de Premio**, **contadores de daño** sobre cada Pokémon, **condiciones especiales activas**, y **flags del turno** (fase actual, acciones ya realizadas: si ya unió Energía, si ya retiró, si ya jugó Partidario, etc.).
- El estado guardado debe ser **suficiente para reconstruir la partida completa** en cualquier momento sin pérdida de información.
- El **registro de acciones (log)** debe ser **completo e inmutable**: cada entrada debe indicar turno, jugador, tipo de acción y resultado. Este log es la fuente para auditoría, revisión de partidas y **reconexión de jugadores desconectados**.

RF-06) Comunicación en tiempo real

Establece los requisitos de sincronización entre clientes y servidor durante la partida. El uso de WebSockets es obligatorio y su implementación es responsabilidad investigativa del equipo.

- El juego debe implementar comunicación **bidireccional en tiempo real** entre clientes y servidor utilizando **WebSockets**. *Este tema **no forma parte del contenido dictado en clase** y no será explicado por la cátedra — su investigación, aprendizaje e implementación son **responsabilidad del equipo**, formando parte del espíritu de investigación del TPI*.
- **Sincronizar el estado** del juego entre ambos jugadores después de cada acción válida, enviando actualizaciones a ambos clientes.
- **Notificar** a los jugadores sobre eventos relevantes: inicio de turno, knockout, toma de carta de Premio, condiciones especiales aplicadas y fin de partida.
- **Manejo robusto de reconexiones:** si un cliente se desconecta, debe poder reconectarse y recibir el **estado actualizado** de la partida para continuar jugando.
- El **backend es la única fuente de verdad** del estado del juego. El frontend **no debe tomar decisiones de juego** ni modificar el estado localmente — solo presenta lo que el servidor indica.

RF-07) Interfaz de usuario

Define la experiencia visual del juego: el lobby, el tablero interactivo, los controles disponibles por fase y la información que debe mostrarse en tiempo real a cada jugador.

- Implementar un **lobby** donde los jugadores puedan crear partidas nuevas y unirse a partidas disponibles.
- Implementar una **pantalla de juego** con tablero interactivo dividido en zonas: zona del oponente (Pokémon Activo, Banca, mazo, cartas de Premio, pila de descarte), zona del jugador (ídem), **zona compartida** para el Estadio activo, y **mano del jugador**.
- Sistema de **drag & drop** para: colocar Pokémon Básicos en Banca, unir Energías a un Pokémon específico, equipar Herramientas, y jugar cartas de Entrenador (con **selección de target** cuando la carta lo requiera).
- Visualización en tiempo real de: **HP actual vs HP máximo**, contadores de daño, Energías unidas, Herramienta equipada, **condición especial activa** (rotación de carta para Dormido, Confundido y Paralizado; marcador para Quemado y Envenenado), **cantidad de cartas de Premio restantes** de cada jugador, y **cantidad de cartas en la mano del oponente** (sin revelar el contenido).
- **Panel de acciones** con botones **habilitados o deshabilitados** según la fase del turno y las acciones ya realizadas: Evolucionar, Unir Energía, Jugar Entrenador, Retirar, Atacar, Finalizar Turno.
- **Log de acciones** visible durante la partida con el historial cronológico de eventos.
- **Notificaciones visuales** para eventos relevantes: inicio de turno, ataques resueltos, knockouts, toma de cartas de Premio, condiciones especiales aplicadas y fin de partida.
- Visualizar y permitir el uso de **Habilidades de Pokémon** (*Abilities*) desde el tablero.
- ***Opcional:*** animaciones para ataques, evoluciones y knockouts.

Requerimientos No Funcionales

La siguiente lista de requerimientos debe cumplirse para que el trabajo se considere completo y correcto, a excepción de los requerimientos opcionales, que serán considerados como parte de la nota normativa.

RNF-01) Rendimiento y optimización

- Las acciones de juego deben tener un tiempo de respuesta promedio **menor a 200ms**.
- La búsqueda de cartas en la API o en el caché local debe responder en **menos de 500ms**.
- El frontend debe renderizar el tablero y procesar acciones interactivas con tiempos de respuesta acordes al juego.
- Optimizar la carga de recursos visuales (imágenes de cartas) utilizando **formatos eficientes**.

RNF-02) Calidad del código y buenas prácticas

- El código backend debe seguir las convenciones de **Java 21** y mantener alta calidad en términos de legibilidad, mantenibilidad y escalabilidad.
- Aplicar los principios **SOLID**, en particular el de responsabilidad única, para garantizar una arquitectura modular y robusta.
- El **Game Engine** debe estar completamente aislado del resto de la aplicación, con componentes independientes y testeables: **RuleValidator**, **DamageCalculator**, **StatusEffectManager**, **VictoryConditionChecker** y **TurnManager**.
- Utilizar **Maven** como herramienta de gestión de dependencias y construcción del proyecto.
- El código frontend debe seguir las mejores prácticas de **Angular 21+**, con TypeScript tipado estrictamente y arquitectura por componentes y servicios, siguiendo la guía de estilo oficial de Angular.
- La interfaz debe ser funcional y usable en **dispositivos de escritorio y tablet**, accesible desde Chrome, Firefox, Safari y Edge.

RNF-03) Pruebas unitarias y cobertura de código

- Implementar pruebas unitarias con **JUnit** y **Mockito** para garantizar la calidad y el correcto funcionamiento del código.
- Lograr una cobertura de código con **JaCoCo** de al menos **80%** sobre el total del proyecto.
- Los componentes **RuleValidator**, **DamageCalculator** y **StatusEffectManager** deben alcanzar una cobertura **mayor al 90%**, dado que concentran la lógica crítica del juego.
- Implementar **tests de integración** que cubran los casos de uso principales: partida completa, mulligan múltiple, evolución, knockout y victoria.
- Implementar al menos **tests E2E** básicos en el frontend cubriendo el flujo completo: crear mazo, unirse a partida y ejecutar un turno.

RNF-04) Diseño de software y patrones de diseño

- Aplicar un enfoque de diseño **modular y bien estructurado**, utilizando patrones de diseño apropiados para mejorar la calidad y mantenibilidad del código.
- Patrones recomendados para este dominio:
  - **State** — para los estados del turno (DRAW, MAIN, ATTACK, BETWEEN_TURNS) y los estados de la partida (WAITING, SETUP, ACTIVE, FINISHED).
  - **Strategy** — para los efectos de cada tipo de carta de Entrenador y para los efectos de cada ataque.
  - **Chain of Responsibility** — para el pipeline de resolución del ataque (los 7 pasos definidos en RF-01c: validación de energía → confusión → selecciones → pre-ataque → modificadores → cálculo de daño → efectos post-daño).
  - **Observer** — para notificar eventos de juego a ambos clientes vía WebSocket (knockout, toma de premio, condición aplicada, fin de turno).
  - **Repository** — para la capa de acceso a datos de cartas, mazos y estado de partida.
  - **Facade** — para el GameEngine, exponiendo una API interna simple al resto de la aplicación y ocultando la complejidad del motor de reglas.
- Se valorará especialmente que el **Game Engine esté completamente aislado e independiente del transporte** (REST/WebSocket), siguiendo el principio de inversión de dependencias.

RNF-05) Seguridad

- Evitar el uso de librerías con **vulnerabilidades críticas o altas** conocidas.
- Todas las acciones de juego deben ser validadas en el **backend**; las validaciones del frontend son complementarias y **no reemplazan** a las del servidor.
- La **mano del oponente nunca debe enviarse al cliente** — solo la cantidad de cartas.
- El **orden del mazo** y el **contenido de las cartas de Premio** deben permanecer ocultos hasta el momento que las reglas indiquen.
- Implementar validaciones en todas las entradas de usuario y un **manejo adecuado de errores** en todas las interacciones con el backend.
- ***Opcional:*** implementar autenticación mediante **JWT** para identificar a los jugadores.

RNF-06) Usabilidad y experiencia de usuario

- La interfaz debe ser **intuitiva y fácil de usar**, con una experiencia de usuario agradable.
- Los mensajes de error deben ser **claros, descriptivos y accionables**. Ejemplo: en lugar de *"Error de validación"*, mostrar *"No puedes atacar: te falta 1 Energía de Fuego para usar este ataque"*.
- Los elementos de la interfaz deben tener **feedback visual inmediato** ante cada acción: botones habilitados/deshabilitados según el contexto, **highlights de targets válidos** (cartas evolucionables, Pokémon que pueden recibir energía, etc.) y retroalimentación ante cada acción relevante (ataque, evolución, toma de premio, fin de turno, etc.).
- Las transiciones entre los diferentes estados del juego deben ser **fluidas**.

Tecnologías Obligatorias

- **Backend:** Java 21+ con Spring Boot 3.x
- **Frontend:** Angular 21+
- **Base de Datos:** PostgreSQL o MySQL
- **API Externa:** pokemontcg.io (v2) — integración con caché local obligatoria
- **Comunicación en tiempo real:** WebSockets *(ver RF-06)*

Forma de trabajo

Para este proyecto los alumnos trabajarán en **grupos de 6 a 7 personas**, lo que fomentará el trabajo en equipo y la colaboración entre los miembros del grupo. Cada equipo será responsable de autogestionarse y organizarse de manera eficiente para llevar a cabo todas las etapas del desarrollo, desde el diseño y la implementación hasta las pruebas y la entrega final.

Una parte fundamental del proyecto será el uso de herramientas de control de versiones, específicamente **Git y GitHub**, utilizando el workflow **GitFlow** aprendido en clases. Se creará un aula virtual (*classroom*) en GitHub específica para este proyecto, donde los equipos podrán acceder a los recursos necesarios, realizar seguimiento del progreso y entregar sus entregables.

Los equipos deberán dividirse las tareas de manera equitativa, asignando responsabilidades a cada miembro en función de sus habilidades y conocimientos, y comunicarse de manera efectiva para mantener a todos los integrantes alineados y contribuyendo activamente.

Entregables obligatorios

- Código fuente en repositorio Git con README.md que incluya: instrucciones de instalación, ejecución local, stack tecnológico utilizado y diagrama de arquitectura.
- Script SQL con el esquema completo de base de datos y datos de ejemplo con migraciones versionadas + seed data.
- Documentación técnica: especificación de API REST (Swagger/OpenAPI), decisiones de diseño justificadas y manual de despliegue.
- Reporte de cobertura de tests generado con **JaCoCo**.

Forma y fechas de entrega

La entrega se realizará a través del aula virtual (*classroom*) de GitHub específica para este proyecto. Todos los requerimientos funcionales y no funcionales obligatorios deberán estar cumplidos en la fecha de entrega para aprobar el proyecto. Las entregas realizadas fuera de término serán consideradas tardías y estarán sujetas a penalizaciones en la evaluación final.

Evaluación y Calificación

Cumplir con los requerimientos establecidos es el criterio mínimo para aprobar el proyecto. Para obtener una evaluación sobresaliente se valorará adicionalmente la calidad del código, la eficiencia del diseño y la implementación, la cobertura de pruebas, la claridad de la documentación y la capacidad de trabajo en equipo y colaboración demostrada durante el desarrollo del proyecto.

- Se evaluará la calidad, complejidad y presentación del trabajo, así como la oportunidad de entrega en fecha.
- Este práctico tiene carácter **obligatorio** y deberá aprobarse con un mínimo del **60%** (al igual que los parciales) para regularizar la materia.
- La calificación alcanzada por el grupo será la **calificación individual** de cada uno de sus integrantes.
- Los requerimientos opcionales otorgan puntaje adicional únicamente si los requerimientos obligatorios están cumplidos. No compensan incumplimientos de requerimientos funcionales o no funcionales obligatorios.

Categoría	Criterio	Puntos
Funcionalidad	Reglas del juego implementadas correctamente según RF-01 a RF-07 y el reglamento oficial XY1	10
	Partida completa jugable de inicio a fin (desde crear mazo hasta victoria)	10
	Validaciones correctas de movimientos (detección de movimientos inválidos en backend)	10
	Condiciones especiales funcionando correctamente (Dormido, Envenenado, Quemado, Confundido, Paralizado), incluyendo incompatibilidades y orden de procesamiento entre turnos	5
	Cálculo de daño correcto (base + debilidad + resistencia + modificadores), según la secuencia definida en RF-01c	5
Subtotal		**40**

Categoría	Criterio	Puntos
Arquitectura y código	Separación de responsabilidades clara: Controller / Service / Repository / Engine	6
	Game Engine bien estructurado con componentes independientes y testeables (RuleValidator, DamageCalculator, StatusEffectManager, VictoryConditionChecker, TurnManager)	6
	Aplicación correcta de patrones de diseño: State, Strategy, Chain of Responsibility, Observer, Repository y Facade, según RNF-04	5
	Código limpio, legible y con nombres descriptivos	4
	Manejo de errores robusto en todas las capas	4
Subtotal		**25**
Base de datos	Modelo de datos correcto con entidades bien diseñadas y relaciones apropiadas	5
	Queries eficientes con índices y sin problemas de N+1	3
	Integridad de datos con constraints y validaciones en base de datos	2
Subtotal		**10**
Frontend	Interfaz funcional y usable: drag & drop, panel de acciones, feedback visual inmediato	6
	Integración con backend: estado sincronizado en tiempo real vía WebSockets	5
	Diseño visual claro y comprensible	2
	Funciona correctamente en desktop y tablet	2
Subtotal		**15**
Testing	Tests unitarios del backend con cobertura ≥ 80% global y ≥ 90% en RuleValidator, DamageCalculator y StatusEffectManager	5
	Tests de integración cubriendo los casos principales: partida completa, mulligan, evolución, knockout y victoria	3
	Al menos un test E2E cubriendo el flujo básico: crear mazo, unirse a partida y ejecutar un turno	2
Subtotal		**10**
	TOTAL OBLIGATORIO	**100**
Opcionales	Mazo temático 100% funcional incluido como *seed data* (válido, jugable, basado en set xy1)	10
	Animaciones y efectos visuales en el frontend (ataques, evoluciones, knockouts)	2
	Sistema de ranking o historial de partidas por jugador	1
	Chat entre jugadores durante la partida	2
Subtotal opcionales		**+15**
	TOTAL	**100 (+ 15 bonus)**

Apéndice — Material de referencia

Los siguientes recursos constituyen la base técnica y reglamentaria del proyecto. Se espera que los equipos los consulten durante el desarrollo.

Reglamento oficial del juego

- **XY Rulebook (español)** — Reglamento oficial del Juego de Cartas Coleccionables Pokémon, edición XY. Define las reglas de juego que deben implementarse en este TPI. Adjunto a este documento.

API de cartas

- **pokemontcg.io (v2)** — API pública para obtener datos oficiales de cartas, sets y expansiones. Documentación: https://docs.pokemontcg.io Endpoint base: https://api.pokemontcg.io/v2/ Set de referencia obligatorio: xy1 (XY Unlimited, 146 cartas)

Cliente digital oficial de referencia

- **Pokémon TCG Live** — Cliente digital oficial del JCC Pokémon. Puede utilizarse como referencia visual para el diseño del tablero, la disposición de zonas de juego y la experiencia de usuario general. Acceso: https://tcg.pokemon.com/es-es/tcgl/

Tecnologías utilizadas — documentación oficial

- **Java 21**
- **Spring Boot 3x**
- **Angular 21+**
- **PostgreSQL**
- **WebSockets con Spring**
- **JUnit 5**
- **Mockito**
- **JaCoCo**
- **Swagger / OpenAPI**

Control de versiones

- **Git**

Atribución-No Comercial-Sin Derivadas
Se permite descargar esta obra y compartirla, siempre y cuando no sea modificado y/o alterado su contenido, ni se comercialice. Universidad Tecnológica Nacional Facultad Regional Córdoba (S/D). Material para la Tecnicatura Universitaria en Programación, modalidad virtual, Córdoba, Argentina.